

Spike: 2**Title: Navigation with Graph****Author:** Pattarapol Tantechasa, 103883220**Goals / deliverables:**

- A game world divided into a larger number of navigation tiles and a correspondingly larger navigation graph structure.
- A path planning system that can create paths for agents based on the dynamic environment, using cost-based heuristic algorithms that account for at least six types of terrain.
- Multiple independent moving agent characters (at least four), each following their own independent paths.
- At least two different types of agents that navigate the world in different ways.

Technologies, Tools, and Resources used:

- Claude: brainstorm ideas, explain code and debugs.
- Visual Studio Code IDE
- Conda: Python environment manager

Tasks undertaken:

1. Download and Install **Visual Studio Code**
2. Install **Python** on your Machine
3. Install **PyGame & Pyglet**
4. Open the main project downloaded from
5. Open terminal and run cd command : **cd Assignments-Labs/Assignment-3/Task-6-Spike-Navigation-with-Graphs/navigation-with-graph**
6. Run: **python main.py map4.txt**

What we found out:**Multi-Agent Independent Pathfinding with Terrain Costs**

The baseline implementation demonstrates successful A* pathfinding across a 20×20 grid with multiple independent agents navigating simultaneously. The key outcomes are:

1. **Dual Graph Architecture for Agent Types:** Ground agents use a weighted graph where each terrain type has different traversal costs (*CLEAR*=1, *MUD*=2-4, *WATER*=5-10, *FOREST*=2-5, *HILL*=3-10, *ICE*=6-11). Flying agents use a flat-cost graph (cost=1 for all non-wall tiles), allowing them to traverse any non-wall terrain at uniform speed. This architectural separation enables fundamentally different movement strategies without code duplication.
2. **Cost-Based Heuristic Pathfinding:** A* search with Euclidean distance heuristic ensures optimal paths while maintaining admissibility. Minimum edge cost (1.0)

validates the heuristic, preventing underestimation. 4 independent agents (2 ground, 2 flying) continuously compute and follow optimal paths.

3. **Continuous Autonomous Movement:** Agents move at constant speed (150 pixels/second) along computed paths, node-by-node. Upon reaching a target, agents autonomously select a new random destination and replan.
4. **Terrain Integration:** 6 terrain types (Clear, Mud, Water, Forest, Hill, Ice) with bidirectional traversal costs. Terrain costs represent real-world difficulty: water and hills are most expensive, clear terrain cheapest. Agents naturally favor easier terrain when available, demonstrating that the cost system drives intelligent pathfinding.

Extension:

Agent Collision and Cost-Based Avoidance

The extension demonstrates that agents can detect collisions, dynamically modify graph costs, and automatically reroute to avoid dangerous areas.

1. **Collision Detection and Agent Respawn:** Collision check runs every frame, comparing agent pixel positions to determine which tile each occupies. When two agents occupy the same tile simultaneously, the “losing” agent is respawned at a random new location. The winning agent continues uninterrupted.
2. **Dynamic Cost Spiking:** Upon collision, the affected tile’s cost is spiked by +20 units. This cost is added directly to edge weights in the A* graph during the next rebuild. The cost increase is proportional where a tile with base cost 2 becomes cost 22, making it extremely undesirable.
3. **Emergent Avoidance Behavior:** All agents (not just the respawned one) automatically replan their paths when the graph changes. Because A* always selects minimum-cost paths, agents naturally route around the dangerous tile. No explicit “avoid tile X” instruction was needed, avoidance emerges from the cost function. This demonstrates the power of A* to adapt to dynamic environmental changes.
4. **Threat Decay and Reintegration:** Dangerous tiles decay at 2.0 cost/second, returning to normal over ~10 seconds. Tile color fades from red (dangerous) back to original (safe) as threat decreases. Agents may route back through formerly dangerous tiles once cost normalizes.

Visual Feedback

- **Red** on collision indicates immediate danger
- **Color fade** over 10 seconds provides clear visual feedback of threat decay
- **Console output** logs collisions, respawns, and repath events for transparency

Relation to Spike Topic:

This extension proves that dynamic graph modification enables adaptive agent behaviour. By treating environmental hazards as temporary cost increases rather than hard obstacles, we create a flexible system where threats are temporary and recoverable, agents adapt automatically without additional code, the same A* algorithm handles both static terrain costs and dynamic threats, and avoidance behavior is emergent, not scripted. This is a key

insight for game AI: modification of the cost function is often more flexible than explicit behavior rules.

